ELUCIDATING INFERENCE IN MULTI-TOKEN PREDICTION

Anonymous authors
Paper under double-blind review

ABSTRACT

Accelerating generation is a key challenge in deployment of modern LLMs. Multi-token prediction provides substantial inference speedups by emitting several tokens per forward pass, but the quality of the resulting text depends critically on how those tokens are accepted, rejected, or resampled. We study this decoding layer directly, treating the multi-token model as a black-box proposal mechanism and identify design choices that best trade throughput against generation quality or alignment to an underlying autoregressive (AR) reference. Together, our improvements yield up to 10 tokens per call across a wide range of applications on several pretrained models, a 200% increase over recent methods, unlocking the full low-latency potential of inference on these models.

caption

1 INTRODUCTION

- AR LLM are the base of modern inference
- many methods to speed up including small draft models. MTP/PTP improves up to 3x.
- design choices have been focused on exactly replicating teacher, if not biased trade-off (example p-threshold)
- proposal of parallizaton (apple, PTP)

Autoregressive decoding emits one token per forward pass. The autoregressive model is therefore both the bottleneck and the reference: it sets the latency of a deployed language model, and it defines what the output is supposed to be, since any method that accelerates it is judged by whether it still produces what that model would have produced.

A long line of work accelerates that reference without replacing it. Speculative decoding drafts with a small autoregressive model and verifies the draft against the target (Leviathan et al., 2023; Chen et al., 2023); later methods make the drafter cheaper by attaching extra decoding heads to the target itself (Stern et al., 2018; Cai et al., 2024; Ankner et al., 2024; Li et al., 2024a), or drop the drafter entirely and retrieve candidate continuations from the prompt (Saxena, 2023). Multi-token prediction pushes hardest on the proposal, training a single model to emit several coordinated tokens per call (Gloeckle et al., 2024; Draxler et al., 2025). Task-average speedups near $2.4\times$, approaching $3\times$ on the most favourable task categories, are now routine on diverse-task benchmarks (Xia et al., 2024).

A multi-token model breaks the one-token-per-pass constraint by proposing several tokens at once, but a proposal is not an output: something must decide which of the proposed tokens are kept. That decision — the *acceptance rule* — is usually treated as a fixed implementation detail, inherited from speculative decoding, where it is chosen to leave the base model's output distribution exactly intact.

That exactness is already being traded away. Medusa (Cai et al., 2024) argues that matching the base model's distribution is typically unnecessary and instead accepts any token the base model finds sufficiently plausible, a rule Hydra (Ankner et al., 2024) inherits. The trade is real and it is large: on our checkpoint, switching the predicate from exact matching to nucleus membership at $\theta = 0.98$ and changing nothing else raises committed tokens per call from 3.63 to 4.30 and wall-clock speedup from $1.02\times$ to $1.70\times$, while the mean probability the base model assigns to the generated tokens

falls by 17 points. Once the proposal mechanism is strong enough to offer many coherent tokens per call, the acceptance rule stops being an implementation detail: it sets the throughput, and it sets how far the generated text drifts from what the base model would have produced. Yet it is chosen by convention and reported by nobody — the standard protocol measures accepted tokens per call and speedup, never faithfulness.

This paper takes the decoding layer as the object of study. We ask the question a practitioner actually faces: *on a single GPU, what is the lowest latency we can achieve that still produces high-quality outputs?* We organize the answer along three axes.

Verification. What counts as a correct token? The strictest rule accepts a proposed token only if the base model, driven by the same randomness, would have emitted exactly that token. Relaxing it to "the base model finds this token plausible" — nucleus membership, a probability threshold, a likelihood ratio — accepts far more tokens per call, at a cost in faithfulness that we measure rather than assume.

Proposal geometry. How many chances to take, and how long each should be. Given a node budget, is it better to propose one long continuation, or fifty short ones? We answer this by modelling the number of correct tokens per call as the extinction time of a branching process, which makes the expected yield separable across depth and the allocation problem exactly solvable.

Parallelization. Verification and the next proposal can run concurrently if one speculates on the verification outcome, which recent work has begun to exploit (Samragh et al., 2025; Draxler et al., 2025). Partial Quadratic Decoding (PQD) (Draxler et al., 2025) does this by preparing a branch for each possible number of correct tokens and spending a budget across those branches. We show that the branch weights and the value of a branch can be read off the same calibrated model that solves the geometry problem, rather than estimated separately.

We make three main contributions.

- **A design space that generalizes existing methods.** We separate multi-token decoding into three independent axes — verification, proposal geometry, and parallelization — and resolve the published methods along them (Table 1). Most are special cases: a chain is a width profile of one, a static draft tree is a profile fixed offline, and EAGLE-2's context-aware tree is a greedy solution to our allocation problem under an independence assumption that we make explicit and then remove. Eleven verification rules sit behind a single code path, so two configurations differ only in the acceptance predicate, and we pair them with a measurement protocol that separates throughput from faithfulness — including a test of whether the generated text is distributionally distinguishable from the base model's (§3).

- **A generative model of N_{cor} that makes proposal geometry solvable.** A zero-inflated Beta-Binomial branching process with shared per-generation frailty predicts per-call accepted-token histograms across three orders of magnitude of proposal width from three parameters ($R^2 \geq 0.976$). Its survival function makes expected yield separable across depth, which turns strand-length selection into a knapsack we solve exactly by dynamic programming — and the same object supplies the branch weights and branch values that Partial Quadratic Decoding needs, replacing two separately calibrated estimates with one (§4–§5.1).

- **Evidence that the standard metric is the wrong one alone.** Across 62 decoding configurations on Spec-Bench, accepted-tokens-per-call, wall-clock speedup, and faithfulness to the base model induce three different orderings. The configurations at the top of the tokens-per-call ranking get there by accepting almost everything, roughly halving the base model's mean token probability; among those that stay close to the base model, more tokens per call often means *slower* generation — wide ensembles reach 4.6 tokens per call at $0.31\times$ autoregressive speed. Only the exact rules are distributionally indistinguishable from the base model (§6).

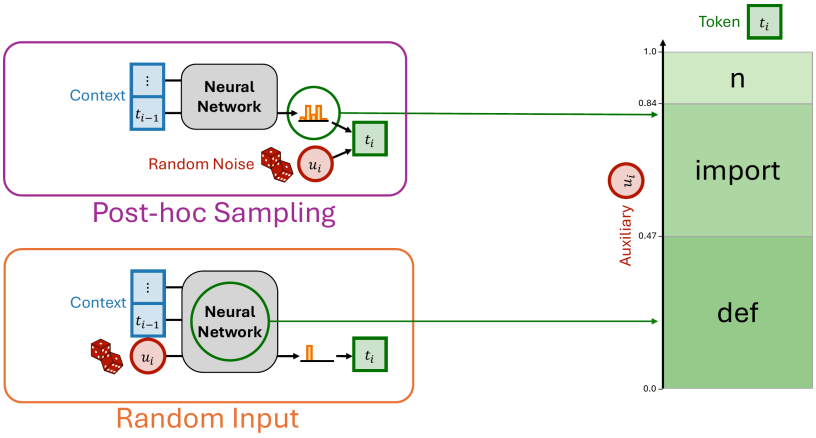


Figure 1: Moving the randomness. **Top:** standard decoding draws a distribution, then samples from it with auxiliary u_i. **Bottom:** PTP feeds u_i in as an input, so the token becomes a deterministic function of context and auxiliaries and future tokens become jointly predictable. **Right:** u_i selects a token by inverse CDF. Reproduced from the PTP framework (Draxler et al., 2025).

2 Background: Multi-Token Proposals and Their Verification

2.1 Auxiliary Variables

We build on Parallel Token Prediction (PTP) (Draxler et al., 2025), which we recap only as far as needed. A standard decoder predicts $P_i := P(t_i \mid t_{<i})$ and then samples $t_i \sim P_i$. The sampling step is where the sequential dependency lives, and it can be made explicit: drawing $u_i \sim \mathcal{U}[0,1]$ and taking the inverse CDF,

$$t_i = \mathrm{Pick}(u_i, P_i) \equiv \min\left\{j : \textstyle\sum_{l \le j} P_{il} > u_i\right\}, \tag{1}$$

makes t_i a *deterministic* function of the context and the auxiliary variable u_i. Since u_i determines t_i given $t_{<i}$, it carries the same information as the token itself, so a model given $u_i, \ldots, u_k$ as inputs can predict t_k without knowing the intervening tokens. This is the basis of PTP: a model $P_\theta(t_k \mid t_{<i}, u_i, \ldots, u_k)$ emits many tokens in one call. Figure 1 illustrates the shift of randomness from post-hoc sampling to a random model input. We use the one-hot variant (O-PTP), whose output probability for the proposed token doubles as a confidence estimate c_k.

2.2 What Counts as a Correct Token

Sharing the auxiliary variables gives verification a strong coupling. A proposed token t_k is *correct* if it equals $\tilde{t}_k = \mathrm{Pick}(u_k, Q_\phi(\cdot \mid t_{<k}))$, the token the base model selects when the *same* auxiliary is applied to its own distribution. Let

$$N_{\mathrm{cor}} = \max\{m \ge 0 : t_{i+j} = \tilde{t}_{i+j} \text{ for all } j < m\} \tag{2}$$

be the longest correct prefix. Because the base model evaluates $Q_\phi(\cdot \mid t_{<k})$ at every position during verification anyway, one token can be sampled from it for free at the first rejection, so a call commits

$$N_{\mathrm{acc}} = N_{\mathrm{cor}} + 1 \tag{3}$$

tokens. Under this rule the committed text is distributed exactly as the base model's, so speedup is free of quality cost; the only question is how large N_{cor} is.

Figure 2 makes the geometry concrete for two tokens. The auxiliary pair (u_1, u_2) ranges over the unit square, and the AR reference partitions that square into regions, one per token pair. A multi-token model induces its own partition; where the two agree, the proposal is correct. PTP's partition follows the reference's curved cell boundaries closely, whereas independent per-position prediction can only produce a product partition of axis-aligned rectangles and therefore mismatches over a large area.

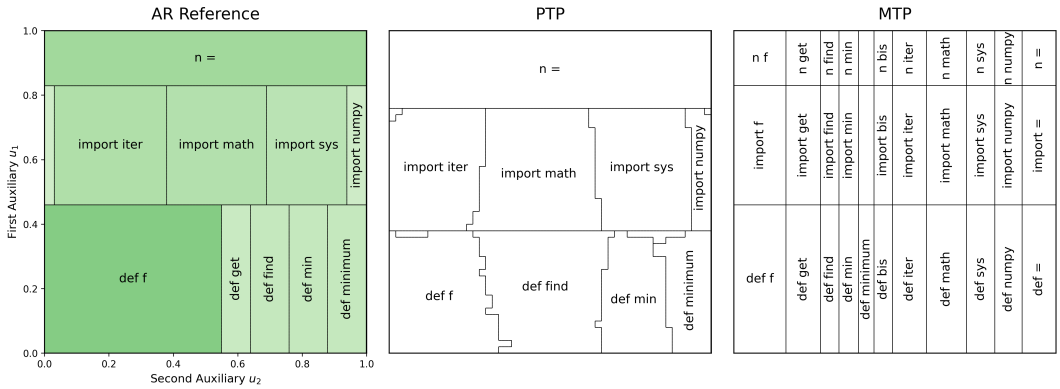


Figure 2: Verification as a partition-matching problem, for the first two generated tokens. The AR reference (left) partitions the auxiliary square into one cell per token pair. PTP (middle) reproduces the curved cell boundaries; independent multi-token prediction (right) can only form a product partition of rectangles. Correctness of a proposal is agreement of the two partitions at the drawn (u_1, u_2). Reproduced from Draxler et al. (2025).

Table 1: The Spec-Bench baselines, resolved along the three axes. "guarantee" is what the method promises about its output: *exact* preserves the base model's distribution, *greedy* reproduces greedy decoding only, *biased* gives up both by design. "overlap" is what runs concurrently with verification. p is the base model's distribution, q / $\hat{p}$ the proposal model's, c_j its confidence at node j, H the entropy. "trained" is what must be fitted before the method can be used: four of the nine need nothing. *SpS trains nothing but requires a second, smaller model of the same family to exist. †SAMD is training-free in its retrieval-only form; its strongest reported configuration falls back to a trained drafter when retrieval misses.

method	proposal source	geometry	verification rule	guarantee	overlap	trained
SpS	small AR model	chain	ratio $\min(1, p/q)$	exact	none	model*
PLD	prompt n-gram copy	chain	argmax match	greedy	none	no
Lookahead	Jacobi, n-gram pool	2D window (W, N)	argmax match	greedy	fused pass	no
Medusa	k independent heads	static tree, offline	$p > \min(\epsilon, \delta e^{-H(p)})$	biased	none	heads
Hydra	dependent heads	static tree, offline	$p > \min(\epsilon, \delta e^{-H(p)})$	biased	none	heads
EAGLE	feature-level AR	static tree	ratio $\min(1, p/\hat{p})$	exact	none	layer
EAGLE-2	feature-level AR	dynamic tree, $\prod_j c_j$	ratio $\min(1, p/\hat{p})$	exact	none	layer
Recycling	top-k adjacency table	static tree, 81/6	argmax match	greedy	none	no
SAMD	suffix automaton	chain + tree	argmax match	greedy	none	no†
this work	PTP, shared u	knapsack profile	any of Table 4	selectable	PQD	LoRA

3 THE DESIGN SPACE OF MULTI-TOKEN DECODING

Any multi-token decoder is characterized three main design choices: how tokens are proposed, what shape the proposal has, and which tokens get accepted into the context. The prediction model is then completed by choosing the pretrained model, the AR reference to align with, and whether any further optimizations, including call parallelization is applied. Existing design choices are described below and existing methods are summarized in Table 1.

3.1 PROPOSAL SOURCE

The framework used to sample multiple tokens in each call, builds core of each multi-token decoder. This usually involves architectural changes, training additional parameters, or finetuning the existing model. Although we will treat the proposal mechanism largely as a black-box, we will briefly describe some possible choices, summarized in Table 2

caption

Trained proposals. The most common choice is to train something small that predicts ahead. Speculative decoding uses a separate, smaller autoregressive model of the same family (Leviathan et al., 2023; Chen et al., 2023): the most expensive source per proposed token, since every token costs a forward pass through the drafter, but the only one whose proposals come with a full distribution q. Cheaper variants attach heads to the frozen target itself — independent heads, one per future position (Stern et al., 2018; Cai et al., 2024), heads made sequentially dependent so each sees what the previous one drafted (Ankner et al., 2024), or a single lightweight decoder layer run over hidden states rather than tokens (Li et al., 2024a;b). These cost a matmul per token instead of a model call, at the price of a training stage and a weaker proposal.

Training-free proposals. A proposal need not come from a model at all. Prompt lookup retrieves a continuation by matching the recent n-gram against the prompt and copying what followed (Saxena, 2023); SAM-Decoding (Hu et al., 2024) generalizes the match to a suffix automaton over the generated text and a reference corpus, at $O(1)$ per step; Token Recycling (Luo et al., 2025) keeps a table of the top-k successors seen for each token and reads candidates out of it. Lookahead decoding instead runs Jacobi iterations on the target model itself and harvests n-grams from the trajectory (Fu et al., 2024). These sources are essentially free per proposed token — a string match or a table lookup — and apply to any model unmodified, which makes them attractive exactly where the answer is partly copyable, such as summarization. What they give up is a calibrated score: none can say how likely a proposed token is to survive verification.

Parallel Token Prediction, our running example. Throughout the paper the proposal comes from PTP (Draxler et al., 2025), which we use because it is coordinated, exposes a confidence, and produces all of its tokens in a single call. Recall from §2 that PTP moves the randomness of sampling from a post-hoc draw to a model input: with $u_i \sim \mathcal{U}[0,1]$ fixed in advance, the token $t_i = \mathrm{Pick}(u_i, P_i)$ is a deterministic function of the context and the auxiliaries, so a model given $u_i, \ldots, u_k$ can predict t_k without knowing the tokens in between. One forward pass over the context plus N_{prop} auxiliary inputs therefore yields N_{prop} proposed tokens. In the one-hot variant we use, the output distribution at each proposed position collapses towards a point mass and its probability doubles as a confidence c_k — the score that the allocation of §5 and the branch weights of §5.1 both consume. Architecturally it is a gated LoRA adapter (Hu et al., 2022) on a frozen base, so disabling the adapter recovers the original model exactly, which is what makes self-verification (Table 4) possible with one set of weights. What distinguishes it from independent multi-token prediction is coordination: because all positions share the same auxiliaries, the proposed tokens are drawn from the joint, and the induced partition of the auxiliary square can follow the reference's curved cell boundaries instead of being a product of per-position rectangles (Figure 2).

What carries over. Three properties of the source, and only these, matter downstream. *Cost per proposed token* spans orders of magnitude between the rows of Table 2, and the allocation of §5 counts nodes uniformly — exactly the assumption that breaks for the training-free sources. *Confidence* is whether a calibrated per-token score exists: the student-confidence rule of Table 4 needs one, and so do the branch weights in §5.1, so the sources marked "–" can use neither. *Coordination* is whether tokens are predicted jointly or apart, which caps what any acceptance rule can recover. Everything else about the source — how it was trained, what architecture it has — the rest of the paper does not need to know.

3.2 PROPOSAL GEOMETRY: HOW MANY CHANCES, AND HOW LONG

Given a budget of C proposal nodes per call, the geometry question is how to spend it: one long continuation, many short ones, or a tree. Every choice is summarized by a *width profile* $W(d)$, the

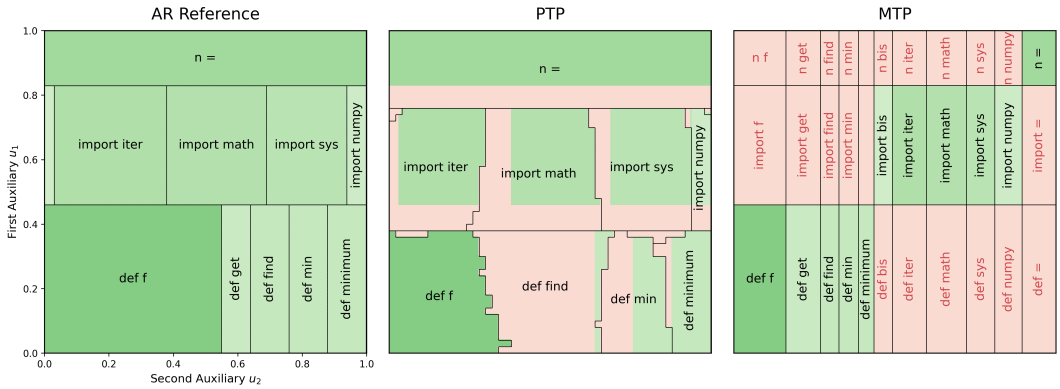
Table 2: Proposal sources. "cost per token" is the marginal cost of one more proposed token; "conf." is whether a calibrated per-token confidence is available (q a full distribution, c_j / c_k a scalar, "per pos." a score that ignores the other proposed positions, "stale" one carried over from an earlier step); "coord." is whether tokens are predicted jointly rather than independently.

proposal source	how a proposal is produced	cost per token	trained	conf.	coord.
small AR model	a separate model, run sequentially	small forward pass	model	q	yes
independent heads	k heads on a frozen backbone	one matmul	heads	per pos.	no
dependent heads	heads see the drafted prefix	k small passes	heads	per pos.	partly
feature-level AR	one decoder layer over hidden states	one layer	layer	c_j	yes
self-refinement	the target iterates on its own guesses	target pass	no	–	via pool
prompt retrieval	n-gram match against the prompt	string match	no	–	copied
automaton retrieval	suffix automaton over past text	$O(1)$ lookup	no	–	copied
token recycling	top-k adjacency table from past steps	table lookup	no	stale	no
independent MTP	one pass, positions predicted apart	one pass	yes	per pos.	no
MTP + auxiliaries	one pass, shared u (PTP)	one pass	LoRA	c_k	yes

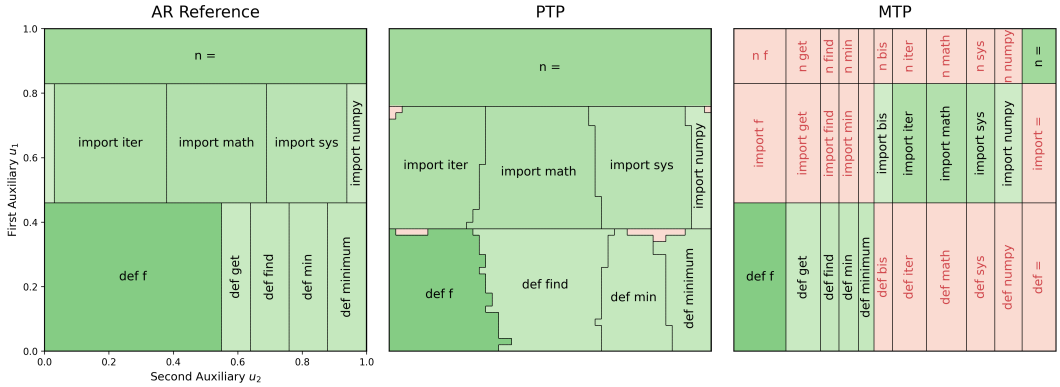
Table 3: Proposal geometries. "correlated" is whether the allocation rule models dependence between sibling strands rather than treating per-node acceptance as independent. $S[w, d]$ is the survival function of §4.

geometry	width profile	allocation chosen by	correlated	used by
chain	$W(d) \equiv 1$	fixed depth	n/a	SpS, PLD, SAMD
2D window	W wide, N deep	fixed (W, N)	no	Lookahead
static tree	fixed, offline	per-head calibration accuracy	no	Medusa, Hydra, EAGLE, Recycling
dynamic tree	per context	greedy on $\prod_j c_j$	no	EAGLE-2
block-iterative	grows by K dependent positions per drafter call	learned rank head	no	Shi et al. (2026)
flat ensemble	$W(d) \equiv K$	fixed K, full depth	no	this work (`standard`)
knapsack profile	nonincreasing $W(d)$	exact DP on $S[w, d]$	**yes**	this work (`optimal`)
block-wise ensemble	K across n blocks	fixed block size	yes	this work (`seq-n`)
sharing tree	non-monotone $W(d)$	open (§5)	partly	–

number of proposal strands still alive at depth d (§5 makes this precise), and Table 3 classifies the options by how that profile is picked. The allocation itself is chosen in one of four ways — fixed offline, greedily from per-node confidences, by a learned head (Shi et al., 2026), or, as we propose in §5, exactly. The column that matters, though, is the last but one: whether the allocation accounts for *correlation* between strands. Failures are not independent — some positions are hard for every strand at once — and §4 shows this is what makes width saturate.



(a) Exact acceptance: keep the proposal only where it *equals* the reference token at the same auxiliary.



(b) Broadened acceptance: keep the proposal wherever the reference finds the token plausible (nucleus membership).

Figure 3: The acceptance rule, not the model, decides what is kept. Both rows show the *same* proposal partitions as Figure 2; green is accepted, pink rejected. Broadening the criterion converts most of the rejected area into accepted area. This is the single largest lever in the design space, and the one that trades faithfulness for throughput most directly.

3.3 Verification: what counts as acceptable

The exact rule of §2.2 is unforgiving: a proposal is rejected whenever the drawn auxiliary falls in the sliver where the two partitions disagree, even if the proposed token is one the base model would have been perfectly happy to emit at a slightly different u. On the example of Figure 3, exact acceptance keeps 63% of the auxiliary square; relaxing the criterion to nucleus membership keeps 99% of it, with the same proposal model and the same proposals.

Table 4 lists the rules. Write q for the proposal model's distribution at a position, p for the base model's, x for the proposed token, u for the shared auxiliary, and $I_p(x)$ for the inverse-CDF interval p assigns to x. Two properties organize the column space. *Does the rule call the base model?* If not (student confidence, unconditional acceptance, self-verification) the rule is nearly free but has no access to the distribution we are trying to stay faithful to. *Does the rule preserve that distribution?* Only the exact and ratio rules do; the rest are deliberate relaxations, and the empirical question is what each one costs. We implement all eleven behind a single code path, so two runs differ only in the predicate. Sharma (2026) studies the same rules analytically, showing that many of their rejection regions are lower level sets of the base distribution; that characterization is deterministic and per-position, complementary to the distributional question we ask here.

Table 4: Verification rules. "base call" is whether the rule consults the model being accelerated; "output" is the resulting guarantee. Rules with no entry in "used by" are, to our knowledge, unexplored in the multi-token setting. We evaluate the first eleven in §6; the last requires a second probe of the base model and is listed but not run.

rule	accept the proposal iff	base call	output	used by
exact (shared u)	$x = \mathrm{Pick}(u, p)$	yes	unchanged	this work
speculative ratio	w.p. $\min(1, p(x)/q(x))$	yes	unchanged	SpS, EAGLE(-2)
argmax match	$x = \arg\max p$	yes	greedy only	PLD, Lookahead, Recycling, SAMD
top-k rank	$\mathrm{rank}_p(x) \le k$	yes	biased	–
nucleus	$\sum_{y:p(y)>p(x)} p(y) < \theta$	yes	biased	–
fixed threshold	$p(x) \ge \theta$	yes	biased	–
entropy-adaptive	$p(x) > \min(\epsilon, \delta e^{-H(p)})$	yes	biased	Medusa, Hydra
inverse-CDF distance	$\mathrm{dist}(u, I_p(x)) \le \theta$	yes	biased ($\theta > 0$)	–
student confidence	$q(x) \ge \theta$	**no**	biased	–
unconditional	always, for the first k	no	biased	–
self-verification	the *student*'s own AR pass accepts x	no	biased	–
self-reflection	a probed p^{refl}, fused with p, accepts x	yes	biased	Wang et al. (2025)

Table 5: Parallelization schemes. "extra compute" is the cost multiplier relative to serial drafting; N_{prop} is the number of proposed tokens and C the node budget.

scheme	what overlaps verification	extra compute	used by
serial	nothing	$1\times$	all baselines but Lookahead
fused pass	guessing, same forward pass	$\sim$ window size	Lookahead
quadratic decoding	one branch per outcome m	$O(N_{\mathrm{prop}}^2)$	PTP
partial quadratic (PQD)	a budgeted subset of branches	$O(C)$	PTP, this work
separate devices	the whole proposal call	2 devices	–

3.4 Parallelization: what overlaps verification

Proposal and verification are naturally serial — the next proposal depends on how much of the last was accepted. Table 5 lists the ways to break that dependency, all of which spend compute to speculate on the verification outcome. This axis is nearly empty in the literature: of the nine baselines, only Lookahead overlaps anything, by fusing guessing into the same forward pass.

The round layout. We make the PQD round concrete for a single strand, as our vLLM implementation runs it. A round verifies a chain of L pending tokens and, in the same call, extends one proposal block per possible outcome: block $c \in \{0, \dots, D-1\}$ is conditioned on exactly c pending tokens being correct and has width $b_c \ge 1$. When verification returns $N_{\mathrm{cor}} = c$, the round commits $\min(c, L) + 1$ tokens, and block $\min(c, L)$ seeds the next round with

$$L' \;=\; \min\left(b_{\min(c,L)} - 1,\; D - 1\right) \tag{4}$$

pending tokens. A *layout* is a depth D together with the block widths $(b_0, \dots, b_{D-1})$, and its cost is the number of positions in the call, $C = D + \sum_c b_c$. Because $L' \le D - 1$, a block wider than D is dead weight, so $1 \le b_c \le D$ and $D \le C - D \le D^2$. The square layout $b_c \equiv D$ is quadratic decoding truncated at depth D; every other layout is a budgeted PQD.

What a layout is worth. The layout acts on acceptance through feedback: the width of the winning block fixes how long the next chain is, and so how much the next round can commit. We compare four models of that feedback. All four are fitted from a single recording: the per-round correct count c_t of a square run at depth 20, in request order. At that depth a round is never capped ($c_t \le 19 < L$), so c_t is the *latent* count — how far the round would have gone given room —

rather than the harvested one. From it we read the marginal $\pi(c)$, its tail $\bar{\pi}(i) = \Pr[c_t \geq i]$, and the transition table $T(c' \mid c)$ between consecutive rounds of a request.

One-step. Pending slot i of block c pays if block c wins and the next round verifies that far. Holding both factors at their marginals makes the objective separable over slots,

$$A_1(b) \;=\; \sum_{c=0}^{D-1} \pi(c) \sum_{i=1}^{b_c-1} \bar{\pi}(i), \tag{5}$$

so the acceptance distribution never responds to the layout.

Markov. The same i.i.d. draws $c_t \sim \pi$, but the chain length is now state: L_{t+1} follows equation 4 from (L_t, c_t). A layout that starves the deep blocks now visibly shortens future chains.

Context. The state is (L_t, c_{t-1}) and the round's count is drawn from $c_t \sim T(\cdot \mid c_{t-1})$ instead of the marginal. This matters because rounds are persistent: c_t has lag-1 autocorrelation 0.31 within a request, and $\mathbb{E}[c_t \mid c_{t-1}]$ runs from 3.1 after a round that accepted nothing to 10.8 after one that accepted nineteen.

Carry. Context, plus the one quantity a square run cannot show. A round offering L slots that would have gone $c > L$ deep commits $L + 1$ tokens and leaves $r = c - L$ known-good tokens ahead of it; the next round's effective count is

$$c_{t+1} \;=\; \max\big(\hat{c},\; r_t\big), \qquad \hat{c} \sim T(\cdot \mid c_t), \qquad r_t = \max(c_t - L_t,\, 0). \tag{6}$$

The latent count does two jobs — how easy the region is and how much of it is left — and in a square run, where depth always equals consumption, the two cannot be told apart. Carrying the residual separates them. It adds no parameter, and a layout that never caps ($r_t \equiv 0$) reduces it to the context model exactly.

The last three models are scored over the same horizon the measurements average over: we propagate probability mass over the state (L_t, c_t, r_t), indexed by the number of tokens generated so far, until 128 tokens have been emitted, and report the ratio of expected committed tokens to expected rounds, $A(b) = \mathbb{E}[\text{tokens}]/\mathbb{E}[\text{rounds}]$. The propagation is exact and runs on a CPU, so any layout can be scored from the recording before it is built.

Throughput. Accepted tokens per round is the algorithm's yield, not its speed. We divide it by a round-cost model fitted by least squares to exclusive-lock timings of seven layouts,

$$\text{tok/s} \;=\; \frac{A(b)}{\tau(C, D)}, \qquad \tau(C, D) \;\approx\; 4.10 + 3.3 \times 10^{-3}\, C + 8.4 \times 10^{-3}\, D \text{ ms}. \tag{7}$$

The fit is valid only for $C \leq 128$: past that width the GEMM tiling steps up, and the square at $D = 11$ ($C = 132$) costs 5.99 ms against 4.56 ms for $D = 10$ ($C = 110$), a jump no linear term can see.

Finding the best layout. Under the one-step model, equation 5 is a sum of per-slot values $\pi(c)\,\bar{\pi}(i)$, each nonincreasing in i, so sorting all slots by value and filling the budget is exact. The feedback models are not separable: a block's width changes the distribution of every later round, which rules out the dynamic program equation 17. We therefore search. For each feasible depth D we start from the even split of $C - D$ over the D blocks, and from a front-loaded split that fills the shallow blocks to D first, and hill-climb by moving one unit of width from block s to block t whenever that raises $A(b)$, subject to $1 \leq b_c \leq D$, until no single transfer helps; the best local optimum over seeds and depths is the pick. To maximize throughput instead, a coarse scan first scores the even split at every (C, D) with $C \leq 128$ by equation 7, and the eight best are refined by the same hill climb. Unlike the knapsack of §5, the result is a local optimum, not a certified one.

3.5 What the framework generalizes

Read across the tables, the baselines are special cases.

Geometry. A chain is $W(d) \equiv 1$. A static tree is a width profile fixed offline — Medusa picks it by greedily adding the node with the highest calibration accuracy, Recycling hand-designs an

imbalanced 81-node, 6-layer shape. EAGLE-2's context-aware tree is an *online* solution to exactly the allocation problem of §5: it scores a node by $V_i = \prod_{t_j \in \mathrm{Path}} p_j \approx \prod_j c_j$ and greedily expands the best. That product is an independence assumption, and Medusa's per-head calibration is the same assumption in offline form. Our model replaces it with a correlation parameter ρ, recovers the independent case as $\rho \to 0$, and solves the allocation exactly rather than greedily.

Verification. The exact and ratio rules are the top two rows of Table 4; PLD, Lookahead, Recycling and SAMD all use argmax match. Medusa and Hydra use the entropy-adaptive threshold $p(x) > \min(\epsilon, \delta e^{-H(p)})$, which is a genuinely different object from a fixed threshold: it accepts liberally where the base model is uncertain and strictly where it is confident. Our exact rule has no baseline analogue, because it needs the auxiliary-variable coupling of §2; speculative sampling reaches the same guarantee by a different mechanism.

Parallelization. Lookahead is the fused-pass row; quadratic decoding (Samragh et al., 2025) and its budgeted variant, Partial Quadratic Decoding (Draxler et al., 2025), are the last two. Kumar et al. (2026) independently arrive at the same structure, preparing speculations for predicted verification outcomes while verification is in flight, and Shen et al. (2026) hedge against likely rejections with parallel branches. Both make explicit a cost our table suppresses: a branch that turns out to be wrong must be rolled back, so the budget buys expected progress rather than guaranteed progress. §5.1 shows that the branch weights and branch values PQD needs are both readable off the same survival function that solves the geometry problem, which removes the two separately calibrated estimates the original formulation requires.

Where the framework does not yet reach. Two gaps are worth stating plainly. First, our width profiles are *nonincreasing*: strands are disjoint, so a node's cost is paid at every depth it spans. Every tree baseline instead shares prefixes, which lets $W(d)$ rise before it falls and changes the cost accounting to $|s_1| + |s_2| - k$ for a shared trunk of length k; we return to this in §5. Second, our budget counts nodes uniformly, whereas retrieval-based proposals (PLD, SAMD, Recycling) cost a string match rather than a forward pass. Both are natural extensions of the same knapsack — a relaxed monotonicity constraint and a per-node cost — but we do not evaluate them here.

4 A MODEL OF THE NUMBER OF CORRECT TOKENS

The verification axis fixes *when* a token is kept. The geometry axis asks how many proposals to make and how long each should be, and answering it requires knowing how N_{cor} behaves. We therefore assume a model of N_{cor} given the number of proposal strands K and the context, and we will need from it only the survival function

$$S[w, d] \;=\; \Pr\left[\, T \geq d \,\middle|\, w \text{ strands} \,\right], \qquad T = N_{\mathrm{acc}} - 1, \tag{8}$$

where T is the number of generations before every strand has failed verification and, following equation 3, $G := N_{\mathrm{acc}} = 1 + T$ is the committed token count per call.

4.1 GEOMETRIC AND BETA-GEOMETRIC BASELINES

The simplest model treats each accepted token as an independent coin flip with success probability p, making T geometric:

$$S[1, d] = p^{\,d-1}, \qquad \Pr[T = d] = (1 - p)\, p^{\,d-1}. \tag{9}$$

For a single strand this is already a good description: Figure 4 fits equation 9 to 5,070 observed calls at $K = 1$ with $R^2 = 0.999$. Allowing the acceptance probability to vary across prompts, $p \sim \mathrm{Beta}(a, b)$, gives the beta-geometric model

$$\Pr[T = d] = \int_0^1 (1 - p)\, p^{\,d-1}\, \mathrm{Beta}(p; a, b)\, \mathrm{d}p = \frac{B(a + d - 1,\, b + 1)}{B(a, b)}, \tag{10}$$

which matters because that variation is real: a likelihood-ratio test against a single pooled p rejects homogeneity decisively ($\chi^2 = 401$ on 99 degrees of freedom, $p \approx 5 \times 10^{-38}$; Figure 11).

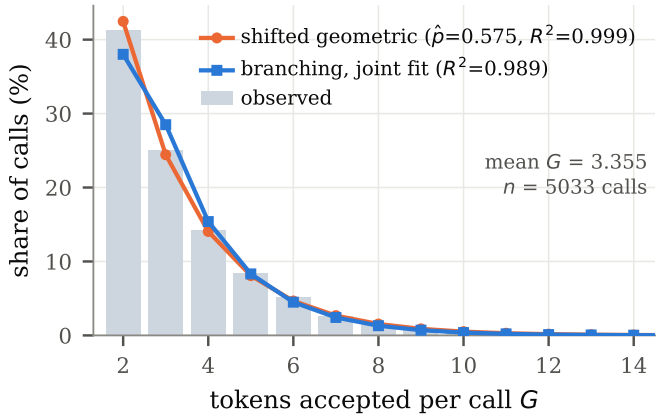


Figure 4: Committed tokens per call at $K = 1$. A shifted geometric equation 9 fitted to this width alone is already excellent; the branching model of §4.2, fitted *jointly* across all five widths, gives up a little accuracy here in exchange for extrapolating across K (Figure 6). Preliminary single-seed run.

4.2 Ensembles need a correlated model

Neither baseline survives the move to $K > 1$ strands. If strands failed independently, total wipeout would vanish geometrically in K; empirically $\Pr[T = 1]$ falls from 41% at $K = 1$ only to 13% at $K = 1000$ and flattens against a floor (Figure 6, left). Some positions are simply hard, and no ensemble width rescues them.

We therefore model the strands as a branching process with a frailty that is *shared* within each generation. At depth i, given $S_{i-1} = m$ surviving strands,

$$\Theta_i \sim \pi_0\,\delta_0 + (1-\pi_0)\,\mathrm{Beta}(\alpha,\beta), \qquad S_i \mid S_{i-1} = m,\ \Theta_i \ \sim\ \mathrm{Binomial}(m,\Theta_i), \tag{11}$$

with $T = \min\{i \geq 1 : S_i = 0\}$. We parameterize by the marginal per-strand success probability p and the within-generation correlation ρ between two strands,

$$\nu = \frac{1-\rho}{\rho}, \qquad \alpha = p\,\nu, \qquad \beta = (1-p)\,\nu. \tag{12}$$

Two features are load-bearing. The shared draw of Θ_i correlates strands within a generation: they tend to fail together, which is why width has diminishing returns. The point mass π_0 at $\Theta = 0$ makes a fraction of positions a guaranteed wipeout regardless of K, producing the observed floor; a pure Beta-Binomial cannot. Marginalizing Θ_i gives a Beta-Binomial transition kernel

$$P[m,j] = (1-\pi_0)\,\mathrm{BetaBin}(j \mid m,\alpha,\beta) + \pi_0\,\mathbb{1}[j=0], \tag{13}$$

with 0 absorbing, from which $S[w,\cdot]$ follows by propagation. Following the structure of the decoder, generation 1 is exempt from zero-inflation and the terminal generation $d_{\max}$ is a forced death, reflecting the hard proposal-length cap. Fitting (π_0, p, ρ) by joint maximum likelihood over all five widths simultaneously gives $\pi_0 = 0.128$, $p = 0.620$, $\rho = 0.789$ on our checkpoint.

5 From survival to allocation: the strand-length knapsack

Strands need not all be the same length. Let L_s be the length budgeted to strand s and define the *width profile*

$$W(d) := \#\{s : L_s \geq d\}, \tag{14}$$

the number of strands still budgeted to reach depth d. The key structural fact is that the shared frailty Θ_d acts conditionally i.i.d. on every strand present in generation d, so any fixed subset of a surviving population is itself Beta-Binomial with the same (α,β) — a thinning property. Consequently the subpopulation that is both alive and still within its own budget at depth d is distributed exactly as

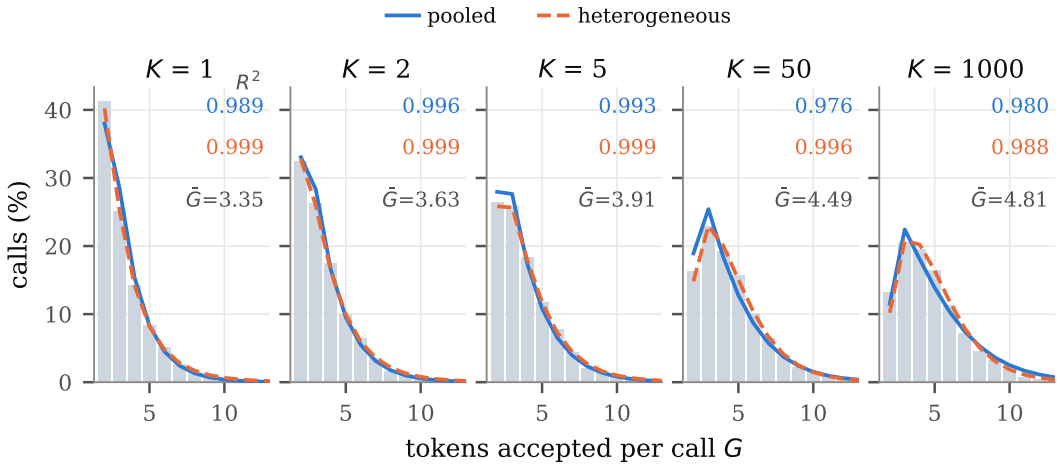


Figure 5: Committed tokens per call against ensemble width. Bars are observed; lines are the branching model equation 11 with a single (π_0, p, ρ) shared across all five panels (*pooled*) and with p integrated over a fitted $\mathrm{Beta}(a, b)$ population (*heterogeneous*). R^2 per curve is printed in the curve's own colour. One three-parameter model tracks the distribution over three orders of magnitude of K.

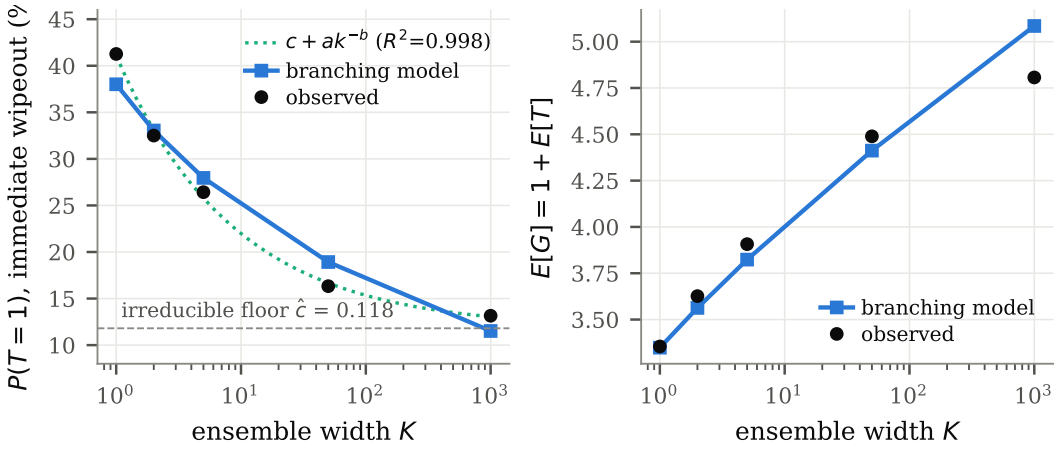


Figure 6: Why width saturates. **Left:** the probability of immediate wipeout decays towards an irreducible floor $\hat{c} = 0.118$, not to zero; a phenomenological $c + aK^{-b}$ captures this, and the branching model produces it from π_0. **Right:** committed tokens per call grow roughly logarithmically in K — a 1000× increase in proposal width buys about 1.5 extra tokens.

a fresh homogeneous run started with $W(d)$ strands, *independently of every other depth*, and the tail-sum identity gives

$$\mathbb{E}[T] = \sum_{d \geq 1} \Pr[T \geq d] = \sum_{d=1}^{d_{\max}} S[W(d), d]. \tag{15}$$

Expected yield is a sum of terms each depending on the width profile at a *single* depth. That separability is what makes the design problem tractable.

Choosing strand lengths under a node budget is then

$$\max_{L_1, \ldots, L_K} \sum_{d=1}^{d_{\max}} S[W(d), d] \qquad \text{s.t.} \qquad \sum_s L_s \leq C, \tag{16}$$

a knapsack whose items are *whole strands*: buying a strand of length L raises the width at every depth $1, \ldots, L$ at once, so its cost is L and its value is the bundled gain across those depths.

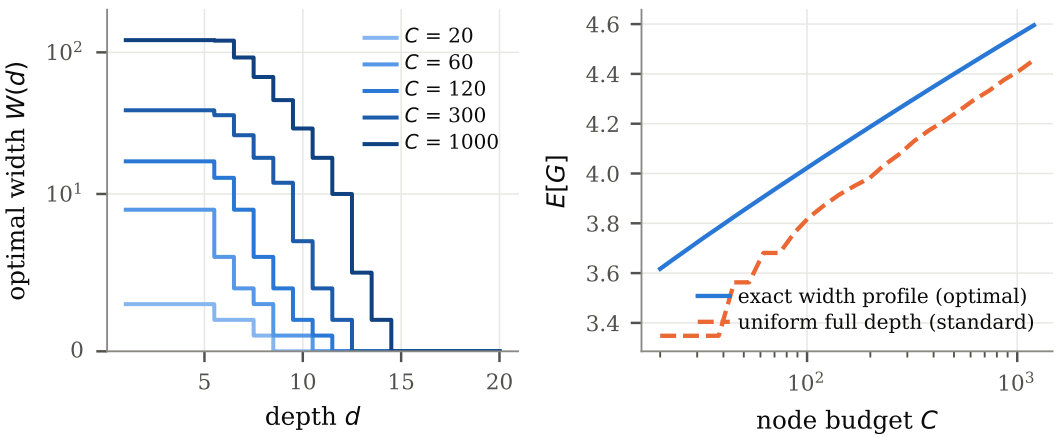


Figure 7: The allocation solved by equation 17. **Left:** optimal width profiles at several node budgets — wide and shallow, never uniform. **Right:** what the exact profile buys over spending the same budget on full-depth strands (the `standard` allocation). The gap is roughly 0.15–0.2 committed tokens per call across two decades of budget.

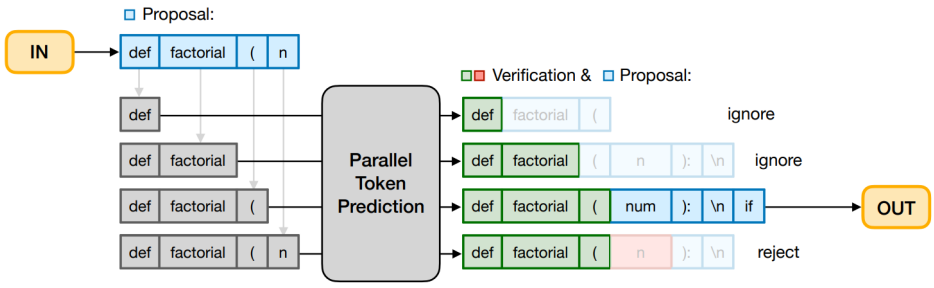


Figure 8: Partial Quadratic Decoding. One branch is prepared per possible number of correct tokens, concurrently with verification; when the base model returns m^*, the matching branch is already extended and the rest are discarded. Reproduced from Draxler et al. (2025).

Exact solution. The feasible width profiles are exactly the nonincreasing integer sequences $W(1) \geq \cdots \geq W(d_{\max}) \geq 0$: any such sequence decomposes into $W(d) - W(d+1)$ strands of length d, by the standard correspondence between nonincreasing sequences and partitions, and its cost $\sum_d W(d)$ equals the sum of those lengths. So equation 16 is a maximization over nonincreasing profiles, solved exactly by a dynamic program over depth:

$$V_d[w_{\text{cap}}, b] \;=\; \max_{0 \leq w \leq \min(w_{\text{cap}},\, b)} \Big\{ S[w, d] + V_{d+1}[w,\, b - w] \Big\}, \qquad V_{d_{\max}+1} \equiv 0, \tag{17}$$

where $V_d[w_{\text{cap}}, b]$ is the best achievable $\sum_{d' \geq d} S[W(d'), d']$ given $W(d) \leq w_{\text{cap}}$ and remaining budget b. Doubling the width bound until the optimum stops touching it makes the result exact at any budget.

5.1 Combining with Partial Quadratic Decoding

Speculative decoding runs proposal and verification in sequence. PQD removes that serialization by speculating on the verification outcome: while the base model verifies N_{prop} proposed tokens, the proposal model prepares a branch for each possible $m \in \{0, \ldots, N_{\text{prop}}\}$, each conditioned on "exactly m of the proposed tokens were correct" (Figure 8). When verification returns m^*, the other branches are discarded and generation continues from the precomputed branch m^*.

Table 6: Fitted survival function $S[w, d] = \Pr[T \geq d]$ for a homogeneous run of w strands, at $(\pi_0, p, \rho) = (0.128, 0.620, 0.789)$. Width has no value at depth 1 and sharply diminishing value thereafter.

	depth d						
w	1	2	3	4	5	6	8
1	1.000	0.620	0.335	0.181	0.098	0.053	0.015
2	1.000	0.670	0.386	0.221	0.125	0.071	0.022
5	1.000	0.720	0.444	0.269	0.161	0.096	0.033
50	1.000	0.811	0.557	0.373	0.245	0.159	0.064

PQD must decide how many continuation tokens B_m to give each branch under a total budget C, which it does by maximizing an expected utility

$$\max_{\sum_m B_m = C} \sum_m \Pr[N_{\text{cor}} = m \mid t]\ H(B_m), \tag{18}$$

solved greedily by marginal gain. As originally formulated, the two ingredients of equation 18 come from different places: the branch weights are estimated by treating errors as independent given the proposal model's confidences, $\Pr[N_{\text{cor}} = m \mid t] \approx (1 - c_{i+m}) \prod_{k<i+m} c_k$, while the reward H is a separate monotone function calibrated on the training set.

§4 supplies both from one object. The branch weights are the increments of the survival function,

$$\Pr[N_{\text{cor}} = m] \ = \ S[K, m] - S[K, m + 1], \tag{19}$$

and the value of giving a branch n tokens is the expected yield of the optimal allocation of those n tokens, i.e. the dynamic program equation 17:

$$H(n) \ = \ \max_{W \text{ nonincreasing},\ \sum_d W(d) \leq n} \ \sum_d S[W(d), d] \ = \ V_1[\infty, n]. \tag{20}$$

Both ingredients are then consistent by construction, calibrated by the same three parameters, and — because H is the *optimal* yield rather than a heuristic proxy — the greedy allocation in equation 18 is comparing branches on the quantity it actually cares about. The independence approximation is retained only where per-position confidences are genuinely more informative than the pooled model, namely in conditioning S on context; §6.4 compares pooled, Bayesian-updated, and learned-head estimates of p empirically.

5.2 Worked example

Table 6 gives the fitted survival function at our checkpoint's $(\pi_0, p, \rho) = (0.128, 0.620, 0.789)$. Two things stand out. $S[w, 1] = 1$ for every $w \geq 1$, so width has *zero* marginal value at depth 1 — a single strand always reaches the first generation. And the returns to width at a fixed depth are strongly concave: at depth 4, going from 1 to 2 strands buys more than going from 5 to 50.

Table 7 turns this into allocations. At a budget of $C = 1000$ nodes — the setting our $K = 50$ runs use — the optimum is 122 strands of mixed length, from 2 of depth 14 down to 29 of depth 6, and it yields 4.555 committed tokens per call against 4.412 for 50 strands at full depth 20. The exact solver reallocates the deep tail of the uniform profile into extra width at shallow depths, where the survival table says it is worth more.

6 Experiments

6.1 Setup

We distil a one-hot PTP student from Vicuna-7B-v1.5 (Zheng et al., 2023) with gated LoRA adapters (Hu et al., 2022) of rank 128 and a binary auxiliary embedding, and evaluate on the first turns of the 13 Spec-Bench categories (Xia et al., 2024) (100 questions), with temperature 0.7, top-k

Table 7: Optimal versus uniform allocation of a node budget, under the fitted model. The profile column reads $\mathrm{depth}^{\mathrm{count}}$, so 6^{29} means 29 strands of length 6. The exact profile trades depth for width and gains ≈ 0.15 committed tokens per call.

budget C	allocation	profile $\mathrm{depth}^{\mathrm{count}}$	strands	$E[G]$
120	exact profile	$11^1\,10^1\,9^2\,8^2\,7^3\,6^4\,5^4$	17	4.067
	uniform full depth	20^6	6	3.873
300	exact profile	$12^2\,11^2\,10^3\,9^5\,8^6\,7^8\,6^{10}\,5^3$	39	4.282
	uniform full depth	20^{15}	15	4.114
1000	exact profile	$14^2\,13^3\,12^5\,11^8\,10^{11}\,9^{17}\,8^{21}\,7^{25}\,6^{29}\,5^1$	122	4.555
	uniform full depth	20^{50}	50	4.412

Table 8: Acceptance rules on Spec-Bench. $\bar{q}$ is the mean per-token probability under the base model, p_{align} the p-value for "generated tokens came from the base distribution". † distribution-preserving by construction. Numbers are preliminary: single-seed runs on 100 Spec-Bench first turns.

acceptance rule	setting	tok./call	ms/tok.	speedup	$\bar{q}$ (%)	PPL	p_{align}
autoregressive†	–	1.00	56.4	1.00×	84.15	1.20	0.251
PTP, single call†	–	2.39	32.7	1.73×	84.51	1.19	0.247
PTP, sequential†	–	3.63	55.3	1.02×	84.71	1.19	0.799
nucleus	$\theta = 0.9$	3.88	35.4	1.59×	74.09	1.40	<0.001
nucleus	$\theta = 0.98$	4.30	33.2	1.70×	67.58	1.56	<0.001
nucleus + ensemble	$\theta = 0.98$, $K = 5$	4.32	57.4	0.98×	82.92	1.22	<0.001
nucleus + ensemble	$\theta = 0.98$, $K = 50$	4.74	140.4	0.40×	82.97	1.22	<0.001
speculative ratio	–	2.35	34.3	1.65×	69.94	1.58	<0.001
speculative ratio	$\kappa = 10$	2.62	31.0	1.82×	70.12	1.57	<0.001
entropy-adaptive	$\epsilon = 0.09$, $\delta = 0.3$	3.02	65.7	0.86×	84.03	1.20	<0.001
entropy-adaptive + PTP	$\epsilon = 0.09$, $\delta = 0.3$	4.40	46.9	1.20×	84.47	1.20	<0.001
teacher threshold	$\theta = 0.1$	3.58	46.0	1.23×	83.16	1.21	<0.001
teacher threshold	$\theta = 0.5$	3.22	52.3	1.08×	85.26	1.18	<0.001
student confidence	$\theta = 0.5$	2.01	69.4	0.81×	41.42	17.03	<0.001
student confidence	$\theta = 0.9$	1.34	92.7	0.61×	64.95	1.77	0.006
accept-k	$k = 2$	2.00	31.7	1.78×	49.29	40.92	<0.001
accept-k	$k = 4$	4.00	16.1	3.50×	47.34	31.99	<0.001
self-verified	exact	3.86	49.9	1.13×	65.92	1.60	<0.001
self-verified + nucleus	$\theta = 0.8$	6.45	39.8	1.42×	24.66	38.70	<0.001
self-verified + ensemble	$\theta = 0.8$, $K = 50$	5.98	111.1	0.51×	73.74	1.40	<0.001

50, top-p 0.9, at most 20 proposed tokens per strand and a total budget of 120 proposed tokens per call, on a single NVIDIA RTX A6000.

We report three families of measure, deliberately kept separate. *Throughput per call* is N_{acc} averaged over calls. *Wall-clock* is milliseconds per generated token, and speedup is the AR run's ms/token divided by the rule's — so it charges every rule for the compute it actually spends. *Faithfulness* is measured under the base model: $\bar{q}$ is the mean per-token probability the base model assigns to the generated tokens (%), PPL the corresponding perplexity, and p_{align} the p-value of a pooled CLT z-test of the null hypothesis that the generated tokens were drawn from the base distribution. A large p_{align} is the operational meaning of "indistinguishable from the base model".

6.2 Acceptance rules: three orderings, not one

Table 8 is the main result. The headline is that the three measures disagree.

Tokens per call is not speed. Sequential exact PTP commits 3.63 tokens per call — more than any single-call rule — yet reaches only 1.02×, because its proposal and verification passes are serialized. The single-call variant commits 2.39 and reaches 1.73×. Ensembles push tokens per call to 4.74 and wall-clock *down* to 0.40×: at $K = 50$ the 1000 proposal nodes cost more than the tokens they

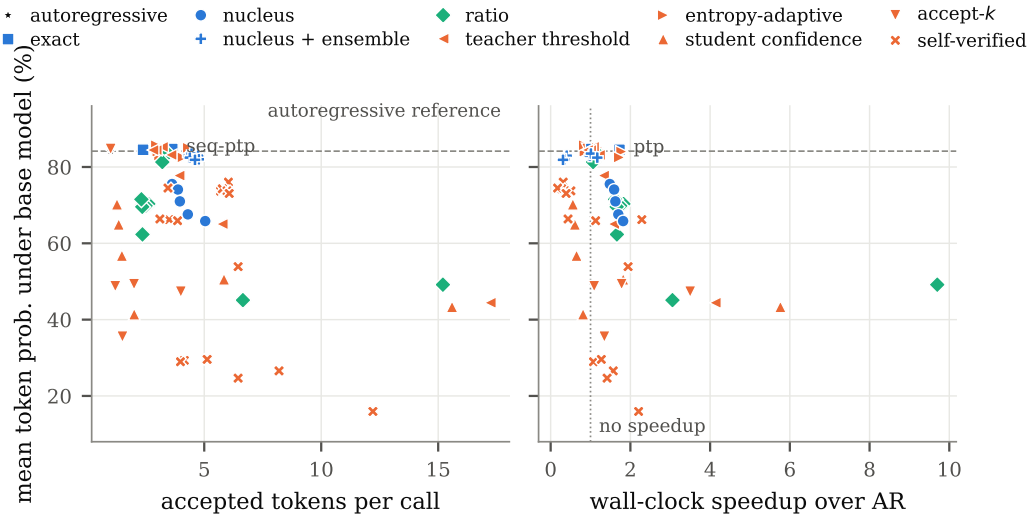


Figure 9: Throughput against faithfulness for all 62 configurations. The two panels share a y-axis but order the rules differently: high tokens per call (left) does not imply wall-clock speedup (right). Only the exact rules sit on the autoregressive reference line.

save. This is precisely the regime PQD (§5.1) exists to fix, and it is invisible if one reports only accepted-tokens-per-call.

Only the exact rules are faithful. The z-test separates the two classes cleanly: $p_{\text{align}} = 0.25$ and 0.80 for exact PTP, versus < 0.001 for every relaxed rule. Broadening the predicate from exact matching to nucleus $\theta = 0.98$ on the same sequential driver gains 0.67 tokens per call and $0.68\times$ in speedup, and costs 17.1 points of $\bar{q}$ ($84.7 \to 67.6$).

Cheap rules are bad rules. Student-confidence thresholding needs no base-model call, and is the worst rule in the table on every axis at once: at $\theta = 0.5$ it commits 2.01 tokens per call, runs *slower* than AR ($0.81\times$), and collapses quality to $\text{PPL} = 17.0$. Accepting k tokens unconditionally is the honest version of the same trade: $3.50\times$ at $k = 4$, with $\text{PPL} = 32.0$.

An anomaly worth flagging. The stochastic ratio rule is distribution-preserving in theory at $\kappa = 1, \varpi = 0$, but we measure $\bar{q} = 69.9$ and $p_{\text{align}} < 0.001$ — a clear deviation. We have not marked it as lossless in Table 8 for that reason. Resolving whether this is an artefact of residual resampling in the truncated top-k space or a genuine implementation defect is left to future work; we report it rather than assume the theory.

6.3 Threshold sweeps

Figure 10 sweeps θ for the three threshold-style families. Base-model-probability thresholding degrades gracefully and monotonically: as $\theta \to 0$ it accepts everything, reaching 17 tokens per call and $4.1\times$, and as θ grows it recovers the base model's quality. Nucleus acceptance occupies a narrow, useful band near $\theta \approx 0.9$–0.98. Student-confidence thresholding is non-monotone and dominated everywhere.

6.4 Ensembles and allocation

Table 9 varies how the per-strand acceptance probability driving the allocation of §5 is estimated: a single pooled p, an online Bayesian update (conjugate Beta-Binomial, or a grid reweighted by the survival likelihood), and a small learned head predicting p from the hidden state. Differences are modest at $K = 5$ and grow at $K = 50$, consistent with the allocation mattering more when there is more budget to misallocate.

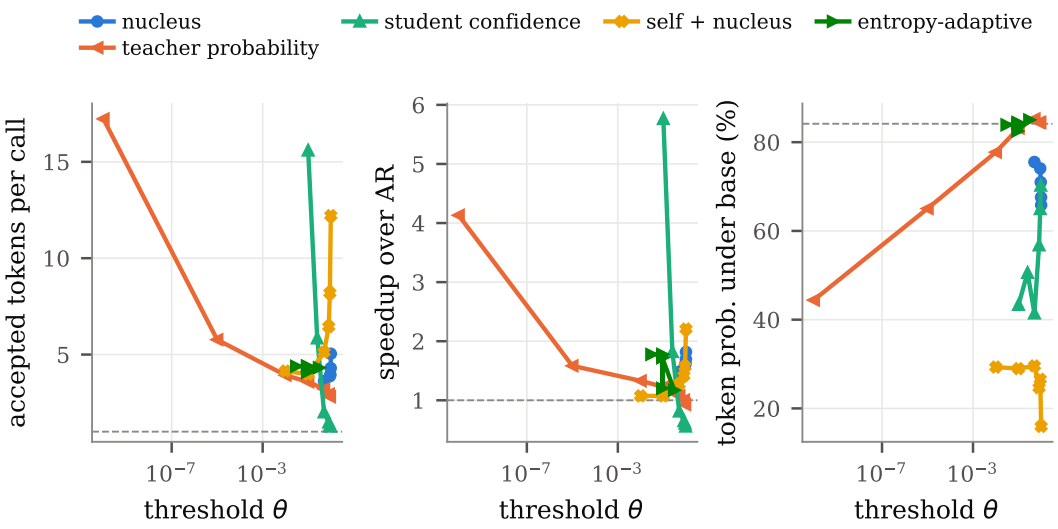


Figure 10: Threshold sweeps. Separate panels rather than twin axes, because the three measures are on unrelated scales. Dashed lines mark the autoregressive reference.

Table 9: Estimating the allocation's acceptance probability. All rows use the nucleus criterion at the stated θ; "allocation" is how p is estimated for the knapsack of §5. Preliminary.

verification	allocation	K	tok./call	ms/tok.	speedup	$\bar{q}$ (%)
verified against the base model, $\theta = 0.98$	uniform	5	4.32	57.4	0.98×	82.92
	Bayes, conjugate	5	4.46	55.9	1.01×	83.52
	Bayes, grid	5	4.38	48.2	1.17×	82.46
	uniform	50	4.74	140.4	0.40×	82.97
	Bayes, conjugate	50	4.77	159.8	0.35×	82.02
	Bayes, grid	50	4.60	182.1	0.31×	81.88
self-verified, $\theta = 0.8$	uniform	50	5.98	111.1	0.51×	73.74
	Bayes, conjugate	50	6.03	179.0	0.32×	76.04
	Bayes, grid	50	5.77	201.8	0.28×	74.29
	learned p-head	50	3.45	325.4	0.17×	74.48

6.5 Does the model predict the data?

Table 10 and Figure 5 validate §4. One three-parameter model, fitted jointly, reproduces the per-call histogram at every width with $R^2 \in [0.976, 0.996]$ and predicts $\mathbb{E}[G]$ to within 0.08 tokens at $K \leq 50$. It degrades at $K = 1000$ (predicting 5.09 against an observed 4.81), which is the extrapolation limit of a depth-uniform (π_0, p, ρ). Integrating p over the fitted population equation 10 improves the fit at every width, by between $+0.3$ and $+2.0$ points of R^2, but not monotonically in K: the largest gain is at $K = 50$ and the gain at $K = 1000$ is smaller than at $K = 1$ (Figure 11). Prompt-level heterogeneity is therefore real and worth modelling, but it is not the dominant source of the pooled model's residual error, and the $K = 1000$ shortfall is not explained by it.

6.6 Self-verification

Dropping the base model entirely and verifying against the proposal model's own autoregressive prediction is attractive — one set of weights, no second model — and it does raise tokens per call (3.86 exact, 6.45 with nucleus $\theta = 0.8$). But faithfulness collapses: $\bar{q} = 65.9$ for exact self-verification and 24.7 with nucleus. This is expected rather than surprising. Self-verification removes the only reference to the distribution we are asking the output to match, so the generation is free to drift, and the metric that detects drift duly does. Self-verification is the right tool when the goal is a fast standalone generator, and the wrong one when the goal is to accelerate a *specific* base model.

Table 10: Model validation across ensemble widths. R^2 of the predicted per-call histogram for the pooled and heterogeneous models, and the likelihood-ratio statistic against a single pooled p (99 d.o.f.).

K	calls	$\bar{G}$ obs.	$E[G]$ model	$\hat{p}_{\text{pool}}$	R^2_{pool}	R^2_{het}	χ^2_{LRT}
1	5070	3.355	3.348	0.592	0.989	0.999	401
2	4630	3.626	3.563	0.563	0.996	0.999	431
5	4321	3.907	3.824	0.523	0.993	0.999	517
50	3870	4.489	4.412	0.449	0.976	0.996	478
1000	3581	4.807	5.085	0.365	0.980	0.988	742

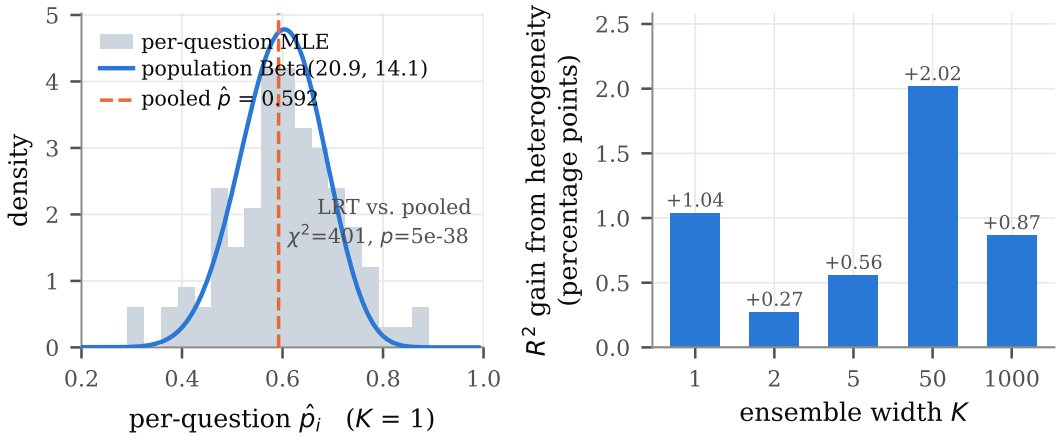


Figure 11: Prompt-level heterogeneity. **Left:** per-question acceptance probabilities at $K = 1$, the fitted population $\text{Beta}(a, b)$, and the pooled estimate; the empirical spread is wider than the population because the per-question MLEs carry estimation noise. **Right:** the resulting gain in histogram fit, recomputed against the same predictions used in Figure 5 and Table 10. The gain is positive at every width but not monotone in K.

7 Related work

Table 1 already resolves the methods we compare against along our three axes, so we note here only what they have in common. Speculative decoding established the proposal-and-verify template (Leviathan et al., 2023; Chen et al., 2023), and the work since has almost entirely gone into the *proposal*: extra decoding heads (Stern et al., 2018; Cai et al., 2024), sequentially dependent heads (Ankner et al., 2024), feature-space drafting with static and then dynamic draft trees (Li et al., 2024a;b), Jacobi-style parallel decoding (Fu et al., 2024), and retrieval from the prompt (Saxena, 2023). Multi-token prediction has also been used as a training objective in its own right (Gloeckle et al., 2024). Verification is correspondingly under-explored: the acceptance rule is inherited, not chosen, with Medusa's entropy-adaptive threshold the notable exception. And the standard benchmark protocol reports accepted-tokens-per-call and speedup (Xia et al., 2024) without any test of distributional faithfulness, which §6.2 shows is where the rules actually differ. Allocation under a draft budget is treated heuristically throughout — EAGLE-2's greedy expansion is the most explicit version, and it assumes independence between sibling nodes. PTP (Draxler et al., 2025) supplies the proposal mechanism we build on, and Partial Quadratic Decoding, which budgets the quadratic expansion of Samragh et al. (2025); the connection between auxiliary variables and inverse autoregressive flows (Kingma et al., 2016) is noted there. Our branching model is a zero-inflated, shared-frailty Galton–Watson process (Athreya & Ney, 1972).

Two concurrent efforts sit closest to this paper. Kumar et al. (2026) break the speculation–verification dependency in the same way Partial Quadratic Decoding does, preparing speculations for predicted verification outcomes; their focus is a systems realization, ours is where the branch weights and branch values should come from. Sharma (2026) gives a deterministic theory of which drafts are accepted, covering greedy, relaxed, top-m and entropy-thresholded rules — the same

taxonomy as Table 4, analyzed per position rather than in distribution. Finally, it is worth noting how narrow the deployed set is relative to the literature: vLLM ships EAGLE, multi-token prediction, draft models, PARD, an MLP proposer, n-gram and suffix decoding, i.e. a handful of proposal sources, all with the acceptance rule inherited rather than chosen.

8 Conclusion

Treating the acceptance rule as a design variable rather than a fixed detail changes the picture of multi-token inference. Accepted-tokens-per-call, wall-clock speedup, and faithfulness to the base model rank decoding rules differently, and reporting only the first — as is common — can recommend a configuration that is both slower and less faithful than the alternative. A three-parameter branching model of the number of correct tokens per call captures the per-call distribution across three orders of magnitude of proposal width, and makes the proposal-geometry question an exactly solvable knapsack whose solution also supplies the branch weights and branch values that Partial Quadratic Decoding needs.

Limitations. All numbers are preliminary, single-seed, and from one model pair (Vicuna-7B and one distilled student) on one GPU; the wall-clock conclusions in particular are hardware- and implementation-specific, and the ensemble rules would be read differently on hardware with spare parallel capacity. We evaluate only first turns of Spec-Bench, which leaves no reference completion, so our quality measures are all relative to the base model rather than to a ground truth; a rule that drifts towards *better* text than the base model's would be penalized by p_{align} exactly as one that drifts towards worse. The fitted (π_0, p, ρ) is depth-uniform and checkpoint-specific, and its extrapolation degrades at $K = 1000$. The ratio-rule anomaly of §6.2 is unresolved.

References

Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.

Krishna B. Athreya and Peter E. Ney. *Branching Processes*. Springer, 1972.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Felix Draxler, Justus Will, Farrin Marouf Sofian, Theofanis Karaletsos, Sameer Singh, and Stephan Mandt. Parallel token prediction for language models. *arXiv preprint arXiv:2512.21323*, 2025.

Yichao Fu, Peter Bai, Hao Zhang, and Ion Stoica. Break the sequential dependency of LLM inference using lookahead decoding. In *International Conference on Machine Learning (ICML)*, 2024.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *International Conference on Machine Learning (ICML)*, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Yuxuan Hu, Ke Wang, Xiaokang Zhang, Fanjin Zhang, Cuiping Li, Hong Chen, and Jing Zhang. SAM decoding: Speculative decoding via suffix automaton. *arXiv preprint arXiv:2411.10666*, 2024.

Durk P. Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.03251.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024a.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024b.

Xianzhen Luo, Yixuan Wang, Qingfu Zhu, Zhiming Zhang, Xuanyu Zhang, Qing Yang, and Dongliang Xu. Turning trash into treasure: Accelerating inference of large language models with token recycling. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

Mohammad Samragh, Arnav Kundu, David Harrison, Kumari Nishu, Devang Naik, Minsik Cho, and Mehrdad Farajtabar. Your LLM knows the future: Uncovering its multi-token prediction potential. *arXiv preprint arXiv:2507.11851*, 2025.

Apoorv Saxena. Prompt lookup decoding. `https://github.com/apoorvumang/prompt-lookup-decoding`, 2023.

Aaryam Sharma. When is a draft accepted? A theory of acceptance in speculative decoding. *arXiv preprint arXiv:2606.30265*, 2026.

Yuhao Shen, Junyi Shen, Quan Kong, Tianyu Liu, Yao Lu, and Cong Wang. SpecBranch: Speculative decoding via hybrid drafting and rollback-aware branch parallelism. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2506.01979.

Weijie Shi, Qiang Xu, Fan Deng, Yaguang Wu, Jiarun Liu, Yehong Xu, Hao Chen, Jia Zhu, Jiajie Xu, Xiangjun Huang, Jian Yang, and Xiaofang Zhou. SpecBlock: Block-iterative speculative decoding with dynamic tree drafting. *arXiv preprint arXiv:2605.07243*, 2026.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Yixuan Wang, Yijun Liu, Shiyu Ji, Yuzhuang Xu, Yang Xu, Qingfu Zhu, and Wanxiang Che. Think before you accept: Semantic reflective verification for faster speculative decoding. *arXiv preprint arXiv:2505.18629*, 2025.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A ALL DECODING CONFIGURATIONS

Table 11 lists every configuration, sorted by wall-clock speedup. Re-run variants of identical configurations are excluded, as are runs marked superseded in the source tree. "gzip" is the compression ratio of the generated text, a model-free repetition proxy; lower means more repetitive.

Table 11: All 62 decoding configurations, sorted by speedup. Preliminary.

run	family	tok./call	ms/tok.	speedup	$\bar{q}$ (%)	PPL	gzip
ratio-p-0.5	ratio	15.20	5.8	9.70×	49.16	240.22	0.226
conf-p-0.1	student confidence	15.59	9.8	5.76×	43.36	124.70	0.125
thresh-0.0000000001	teacher threshold	17.23	13.6	4.13×	44.41	96.82	0.318
first_4	accept-k	4.00	16.1	3.50×	47.34	31.99	0.310
ratio-p-0.1	ratio	6.65	18.5	3.06×	45.10	19.04	0.520
ptp_self	self-verified	3.50	24.7	2.29×	66.21	1.61	0.550
seq-ptp-topp-self-0.99-tok	self-verified	12.21	25.6	2.21×	15.94	91.95	0.471
seq-ptp-self-aux	self-verified	6.45	29.0	1.94×	53.88	5.45	0.479
ratio-10000	ratio	2.61	30.9	1.83×	70.33	1.57	0.721
conf-p-0.25	student confidence	5.84	31.0	1.82×	50.57	57399.95	0.363
tmp_seq-ptp-topp-0.99	nucleus	5.04	31.0	1.82×	65.83	1.67	0.641
ratio-10	ratio	2.62	31.0	1.82×	70.12	1.57	0.717
first_2	accept-k	2.00	31.7	1.78×	49.29	40.92	0.515
seq-ptp-entr-0.09-d0.1	entropy-adaptive	4.40	31.7	1.78×	84.04	1.20	0.674
seq-ptp-entr-0.03-d0.3	entropy-adaptive	4.39	31.7	1.78×	83.91	1.20	0.673
ratio-2	ratio	2.49	32.3	1.75×	69.81	1.58	0.727
ptp	exact	2.39	32.7	1.73×	84.51	1.19	0.685
seq-ptp-entr-0.09-d1.0	entropy-adaptive	4.08	32.8	1.72×	82.51	1.22	0.615
tmp_seq-ptp-topp-0.98	nucleus	4.30	33.2	1.70×	67.58	1.56	0.671
ratio-p-0.01	ratio	2.36	33.9	1.66×	62.33	1.83	0.736
ratio-p-0.001	ratio	2.35	34.1	1.66×	69.56	1.59	0.728
ratio	ratio	2.35	34.3	1.65×	69.94	1.58	0.726
tmp_seq-ptp-topp-0.95	nucleus	3.96	34.7	1.62×	70.99	1.46	0.687
ratio_g	ratio	2.31	34.9	1.62×	71.52	1.50	0.686
tmp_seq-ptp-topp-0.9	nucleus	3.88	35.4	1.59×	74.09	1.40	0.705
thresh-0.00001	teacher threshold	5.77	35.6	1.58×	65.05	1.71	0.588
seq-ptp-topp-self-0.9-tok	self-verified	8.20	35.9	1.57×	26.57	63.95	0.475
tmp_seq-ptp-topp-0.5	nucleus	3.62	38.0	1.48×	75.53	1.39	0.707
tmp_seq-ptp-topp-self-0.8-tok	self-verified	6.45	39.8	1.42×	24.66	38.70	0.537
first_1.5	accept-k	1.50	41.9	1.35×	35.54	18.19	0.742
thresh-0.01	teacher threshold	3.93	42.4	1.33×	77.75	1.31	0.671
entr-0.03-d0.3	entropy-adaptive	3.09	44.1	1.28×	82.96	1.22	0.680
tmp_seq-ptp-topp-self-0.5-tok	self-verified	5.12	44.4	1.27×	29.58	52.51	0.535
thresh-0.1	teacher threshold	3.58	46.0	1.23×	83.16	1.21	0.672
seq-ptp-entr-0.09-d0.3	entropy-adaptive	4.40	46.9	1.20×	84.47	1.20	0.674
seq-ratio	ratio	3.43	48.1	1.17×	83.53	1.21	0.689
seq-ptp-entr-0.3-d0.3	entropy-adaptive	4.32	48.1	1.17×	85.03	1.19	0.679
tmp_seq-ptp-topp-choice-5-0.98-bayes-p	nucleus + ensemble	4.38	48.2	1.17×	82.46	1.23	0.658
seq-ptp-self-tok	self-verified	3.86	49.9	1.13×	65.92	1.60	0.555
first_1.2	accept-k	1.20	51.6	1.09×	48.75	2.90	0.722
thresh-0.5	teacher threshold	3.22	52.3	1.08×	85.26	1.18	0.678
tmp_seq-ptp-topp-self-0.01-tok	self-verified	4.15	52.4	1.08×	29.31	21.01	0.569
tmp_seq-ptp-topp-self-0.1-tok	self-verified	3.98	52.6	1.07×	28.96	24.87	0.564
seq-ratio_g	ratio	3.21	53.0	1.06×	81.29	1.25	0.687
seq-ptp	exact	3.63	55.3	1.02×	84.71	1.19	0.687
tmp_seq-ptp-topp-choice-5-0.98-bayes-conjugate	nucleus + ensemble	4.46	55.9	1.01×	83.52	1.21	0.660
thresh-0.8	teacher threshold	2.95	56.1	1.00×	84.43	1.19	0.684
ar	autoregressive	1.00	56.4	1.00×	84.15	1.20	0.705
tmp_seq-ptp-topp-choice-5-0.98	nucleus + ensemble	4.32	57.4	0.98×	82.92	1.22	0.671
thresh-0.9	teacher threshold	2.81	60.8	0.93×	84.44	1.19	0.687
first_1	accept-k	1.00	61.4	0.92×	84.73	1.19	0.686
entr-0.09-d0.3	entropy-adaptive	3.02	65.7	0.86×	84.03	1.20	0.682
entr-0.3-d0.3	entropy-adaptive	2.95	67.0	0.84×	85.70	1.18	0.683
conf-p-0.5	student confidence	2.01	69.4	0.81×	41.42	17.03	0.512
conf-p-0.8	student confidence	1.48	86.8	0.65×	56.78	2.24	0.675
conf-p-0.9	student confidence	1.34	92.7	0.61×	64.95	1.77	0.679
conf-p-0.95	student confidence	1.27	101.1	0.56×	70.23	1.58	0.688
tmp_seq-ptp-topp-choice-50-self-0.8-tok	self-verified	5.98	111.1	0.51×	73.74	1.40	0.483
tmp_seq-ptp-topp-choice-50-self-0.5-tok	self-verified	5.70	116.6	0.48×	73.70	1.41	0.502
tmp_ptp-topp-choice-50-self-0.5	self-verified	3.10	129.3	0.44×	66.38	1.64	0.484
tmp_seq-ptp-topp-choice-50-0.98	nucleus + ensemble	4.74	140.4	0.40×	82.97	1.22	0.680
tmp_seq-ptp-topp-choice-50-self-0.98-tok	self-verified	6.07	145.2	0.39×	73.06	1.41	0.490
tmp_seq-ptp-topp-choice-50-0.98-bayes-conjugate	nucleus + ensemble	4.77	159.8	0.35×	82.02	1.23	0.685
tmp_seq-ptp-topp-choice-50-self-0.8-bayes-conjugate-tok	self-verified	6.03	179.0	0.32×	76.04	1.36	0.456
tmp_seq-ptp-topp-choice-50-0.98-bayes-p	nucleus + ensemble	4.60	182.1	0.31×	81.88	1.24	0.691
tmp_seq-ptp-topp-choice-50-self-0.8-bayes-p-tok	self-verified	5.77	201.8	0.28×	74.29	1.39	0.485
tmp_seq-ptp-topp-choice-50-self-0.8-phead-tok	self-verified	3.45	325.4	0.17×	74.48	1.39	0.512